Java source code

```
// ============================================================
//  Step 1 — Verify CDK Installation
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;

import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;

public class Step1_VerifyInstall {

    public static void main(String[] args) {

        // ── Check 1: CDK version ──────────────────────────────────────
        // CDK embeds its version in the manifest; read it reflectively
        String cdkVersion = "unknown";
        try {
            Package pkg = Class.forName("org.openscience.cdk.CDK").getPackage();
            if (pkg != null && pkg.getImplementationVersion() != null) {
                cdkVersion = pkg.getImplementationVersion();
            } else {
                // Fallback: check if core class loads
                Class.forName("org.openscience.cdk.silent.SilentChemObjectBuilder");
                cdkVersion = "2.x (loaded successfully)";
            }
        } catch (ClassNotFoundException e) {
            System.err.println("  ERROR: CDK not found on classpath.");
            System.err.println("  -> Download cdk-2.9.jar from:");
            System.err.println("     https://github.com/cdk/cdk/releases");
            System.err.println("  -> Place in the project lib/ folder.");
            System.err.println("  -> In NetBeans: right-click project -> Properties ->
Libraries -> Add JAR");
            return;
        }
        System.out.println("  CDK version   : " + cdkVersion);

        // ── Check 2: Parse a real SMILES string ──────────────────────
        // RDX (cyclotrimethylenetrinitramine) — one of the most important
        // secondary explosives. Formula C3H6N6O6, det. velocity ~8750 m/s.
        // Klapötke (2019), Table 1.1

        String rdxSmiles = "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]";

        try {
            IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
            SmilesParser parser        = new SmilesParser(builder);
            IAtomContainer rdxMol      = parser.parseSmiles(rdxSmiles);

            // Add implicit hydrogens so atom count matches molecular formula
            AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(rdxMol);

            int atomCount = rdxMol.getAtomCount();
            System.out.println("  SMILES parse  : OK — RDX parsed successfully");
            System.out.println("  Heavy atoms   : " + atomCount + " (expected: 15 heavy
atoms)");

        } catch (Exception e) {
            System.err.println("  ERROR during SMILES parsing: " + e.getMessage());
```

```
            return;
        }

        // ── Check 3: Confirm key CDK classes are accessible ──────────────────
        String[] requiredClasses = {
            "org.openscience.cdk.smiles.SmilesParser",
            "org.openscience.cdk.smiles.SmilesGenerator",
            "org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator",
            "org.openscience.cdk.fingerprint.CircularFingerprinter",
            "org.openscience.cdk.qsar.descriptors.molecular.WeightDescriptor",
        };

        System.out.println("\n  Checking required CDK classes:");
        boolean allOk = true;
        for (String className : requiredClasses) {
            try {
                Class.forName(className);
                // Shorten display name for readability
                String shortName = className.substring(className.lastIndexOf('.') + 1);
                System.out.printf("    %-35s  [OK]%n", shortName);
            } catch (ClassNotFoundException e) {
                System.out.printf("    %-35s  [MISSING]%n", className);
                allOk = false;
            }
        }

        // ── Summary ──────────────────────────────────────────────────────────
        System.out.println();
        if (allOk) {
            System.out.println("  All CDK imports OK — environment is ready.");
            System.out.println("  Java version  : " +
System.getProperty("java.version"));
            System.out.println("  OS            : " + System.getProperty("os.name"));
            System.out.println("\n  Proceed to Step 2: SMILES Basics");
        } else {
            System.out.println("  Some classes missing — ensure you are using cdk-
2.9.jar");
        }

    }
}
```

Output: Appended in the final java class Main.

```
// ============================================================
//  Step 2 — SMILES Basics
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.exception.InvalidSmilesException;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Step 2 — SMILES Basics
 *
```

```
 * SMILES (Simplified Molecular Input Line Entry System) encodes a molecule as a
 * plain text string.
 *
 * SMILES notation rules: - Atoms: C, N, O, H, S, P (element symbols) - Single
 * bond: default (no symbol) - Double bond: = e.g. C=O (formaldehyde) - Triple
 * bond: # e.g. C#N (cyanide) - Branches: () e.g. CC(=O)O (acetic acid) - Rings:
 * digits e.g. c1ccccc1 (benzene) - Charges: [NH4+], [O-] - Aromatic: lowercase
 * c, n, o
 *
 * Reference: Weininger, D. (1988). SMILES, a chemical language and information
 * system. J. Chem. Inf. Comput. Sci., 28(1), 31-36.
 * https://doi.org/10.1021/ci00057a005
 *
 */
public class Step2_SMILESBasics {

    private static final Map<String, String> MOLECULES = new LinkedHashMap<>();

    static {
        MOLECULES.put("Methane", "C");
        MOLECULES.put("Ethanol", "CCO");
        MOLECULES.put("Acetic acid", "CC(=O)O");
        MOLECULES.put("Benzene", "c1ccccc1");
        MOLECULES.put("Nitromethane", "C[N+](=O)[O-]");
        MOLECULES.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        MOLECULES.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
        MOLECULES.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-
])C2[N+](=O)[O-]1");
        MOLECULES.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-
])CO[N+](=O)[O-]");
        MOLECULES.put("Ammonium nitrate", "[NH4+].[O-][N+]([O-])=O");
        MOLECULES.put("Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-
]");
        MOLECULES.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
    }

    public static void main(String[] args) {

        // ── Parse and display each molecule ──────────────────────────────────
        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);

        // Suppress CDK logging noise to keep console output clean
        System.setProperty("cdk.logging.level", "ERROR");

        System.out.printf("  %-22s  %-10s  %-8s  %s%n",
                "Compound", "Formula", "Atoms", "Parse Status");
        System.out.println("  " + "-".repeat(72));

        int okCount = 0;
        int errorCount = 0;

        for (Map.Entry<String, String> entry : MOLECULES.entrySet()) {
            String name = entry.getKey();
            String smiles = entry.getValue();
            try {
                IAtomContainer mol = parser.parseSmiles(smiles);
                // Configure atom types so implicit H counts are correct
                AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);
                // Get molecular formula
                IMolecularFormula formula
                        = MolecularFormulaManipulator.getMolecularFormula(mol);
                String formulaStr
                        = MolecularFormulaManipulator.getString(formula);
                int atomCount = mol.getAtomCount();
                System.out.printf("  %-22s  %-10s  %-8d  OK%n",
```

```
                        name, formulaStr, atomCount);
                okCount++;
            } catch (InvalidSmilesException e) {
                System.out.printf("  %-22s  %-10s  %-8s  ERROR: %s%n",
                        name, "?", "?", e.getMessage());
                errorCount++;
            } catch (Exception e) {
                System.out.printf("  %-22s  %-10s  %-8s  ERROR: %s%n",
                        name, "?", "?", e.getClass().getSimpleName());
                errorCount++;
            }
        }
        // ── Summary ──────────────────────────────────────────────────────────
        System.out.println("  " + "-".repeat(72));
        System.out.printf("  Parsed successfully: %d / %d molecules%n",
                okCount, MOLECULES.size());

    }
}
```

Output: Appended in the final java class Main.

```
// ============================================================
//  Step 3: Name to SMILES (PubChem)
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;

public class Step3_NameToSMILES {

    private static final String PUBCHEM_BASE
            = "https://pubchem.ncbi.nlm.nih.gov/rest/pug/compound/name/";

    private static final String PROPERTIES
            =
"IsomericSMILES,CanonicalSMILES,MolecularFormula,MolecularWeight,InChI,InChIKey";

    public static class CompoundInfo {

        public final String canonicalSMILES;
        public final String molecularFormula;
        public final String molecularWeight;
        public final String inchi;
        public final String inchiKey;
        public final int cid;

        public CompoundInfo(String canonical, String formula, String mw, String inchi,
String inchiKey, int cid) {
            this.canonicalSMILES = canonical;
            this.molecularFormula = formula;
```

```
            this.molecularWeight = mw;
            this.inchi = inchi;
            this.inchiKey = inchiKey;
            this.cid = cid;
        }
    }

    public static CompoundInfo parsePubChemJson(String body) {
        // Updated keys matching your raw JSON layout string properties precisely
        String smiles = extractJsonString(body, "SMILES");
        String formula = extractJsonString(body, "MolecularFormula");
        String mw = extractJsonString(body, "MolecularWeight");
        String inchi = extractJsonString(body, "InChI");
        String inchiKey = extractJsonString(body, "InChIKey");
        int cid = extractCID(body);
        if (smiles == null || smiles.isEmpty()) {
            return null;
        }
        return new CompoundInfo(smiles, formula, mw, inchi, inchiKey, cid);
    }

    private static String extractJsonString(String json, String key) {
        String pattern = "\"" + key + "\":";
        int start = json.indexOf(pattern);
        if (start < 0) {
            return null;
        }
        start += pattern.length();
        while (start < json.length() && (json.charAt(start) == ' ' ||
json.charAt(start) == '"')) {
            start++;
        }
        int end = json.indexOf("\"", start);
        if (end < 0) {
            return null;
        }
        return json.substring(start, end);
    }

    private static int extractCID(String json) {
        String pattern = "\"CID\":";
        int idx = json.indexOf(pattern);
        if (idx < 0) {
            return -1;
        }
        idx += pattern.length();
        while (idx < json.length() && (json.charAt(idx) == ' ' || json.charAt(idx) ==
':')) {
            idx++;
        }
        int end = idx;
        while (end < json.length() && Character.isDigit(json.charAt(end))) {
            end++;
        }
        try {
            return Integer.parseInt(json.substring(idx, end));
        } catch (NumberFormatException e) {
            return -1;
        }
    }

    static String moldata = "";
    static String failed_mol = "";
    static IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    static SmilesParser parser = new SmilesParser(builder);
    public static CompoundInfo nameToSMILES(String compoundName) {
        HttpURLConnection connection = null;
```

```
        try {
            String encoded = URLEncoder.encode(compoundName,
StandardCharsets.UTF_8.name());
            String urlStr = PUBCHEM_BASE + encoded + "/property/" + PROPERTIES +
"/JSON";
            URL url = new URL(urlStr);
            connection = (HttpURLConnection) url.openConnection();
            connection.setConnectTimeout(15000); // 15 seconds
            connection.setReadTimeout(20000);    // 20 seconds
            int responseCode = connection.getResponseCode();
            System.out.println("Searching Pubchem: " + compoundName);
            if (responseCode == 404) {
                failed_mol += compoundName.replaceAll("\\s+", "+") + "\n";
                return null;
            }
            if (responseCode != 200) {
                System.out.printf("  HTTP %d for: %s%n", responseCode, compoundName);
                return null;
            }

            StringBuilder responseBody = new StringBuilder();
            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(connection.getInputStream(),
StandardCharsets.UTF_8))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    responseBody.append(line);
                }
            }
            String body = responseBody.toString();
            CompoundInfo info = parsePubChemJson(body);
            if (info != null) {

                moldata += compoundName + "\t";
                moldata += info.cid + "\t";
                moldata += info.molecularFormula + "\t";
                moldata += info.molecularWeight + "\t";
                moldata += info.canonicalSMILES + "\t";
                moldata += info.inchi + "\t";
                moldata += info.inchiKey + "\t";

                try {
                    IAtomContainer mol = parser.parseSmiles(info.canonicalSMILES);
                    AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                    moldata += mol.getAtomCount() + "\t";
                    moldata += mol.getBondCount() + "\n";
                } catch (Exception e) {

                }
            } else {
                System.out.println("Extraction failed! SMILES data field not
located.");
            }
            return new CompoundInfo(info.canonicalSMILES, info.molecularFormula,
info.molecularWeight, info.inchi, info.inchiKey, info.cid);
        } catch (Exception e) {
            return null;
        } finally {
            if (connection != null) {
                connection.disconnect();
            }
        }
    }

    private static String extractField(String source, String regex) {
        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(source);
```

```
        if (matcher.find()) {
            return matcher.group(1);
        }
        return "?";
    }

    private static String extractJsonValue(String json, String key) {
        String pattern = "\"" + key + "\":";
        int start = json.indexOf(pattern);
        if (start < 0) {
            return null;
        }
        start += pattern.length();
        if (start < json.length() && json.charAt(start) == '"') {
            start++;
            int end = json.indexOf("\"", start);
            return end < 0 ? null : json.substring(start, end);
        }
        int end = start;
        while (end < json.length()
                && (Character.isDigit(json.charAt(end)) || json.charAt(end) == '.')) {
            end++;
        }
        return json.substring(start, end);
    }

    private static String generateRepeatedChar(char ch, int count) {
        char[] array = new char[count];
        java.util.Arrays.fill(array, ch);
        return new String(array);
    }

    public static void main(String[] args) {


System.out.println("============================================================");
        System.out.println("  Name to SMILES via PubChem REST API (Java 1.8)");
        System.out.println("  Requires: active internet connection");

System.out.println("============================================================\n");
        disableSSLVerification();

        String[] names = {
            "TNT",
            "RDX",
            "HMX",
            "PETN",
            "nitromethane",
            "ammonium nitrate",
            "picric acid",
            "tetryl",
            "TATB",
            "FOX-7",};

        for (String name : names) {
            try {
                Thread.sleep(250);
            } catch (InterruptedException ignored) {
            }
            CompoundInfo info = nameToSMILES(name);
        }

        System.out.print("Compound Name\tCID\tFormula\tMol
Wt\tSMILES\tInChI\tInChIKey\tAtom count\tBond Count\n" + moldata);
        System.out.println("\n\n======Not found in PubChem:=====\n" + failed_mol);
        System.out.println("\nCitation: \n1) Practical Chemoinformatics,2014,
M.Karthikeyan, Renu Vyas; \n2) Kim et al. (2023). PubChem 2023 update.");
```

```
        System.out.println("  Nucleic Acids Res., 51, D1373-D1380.");
        System.out.println("======= ");
    }

    // Add this method inside your class
    private static void disableSSLVerification() {
        try {
            javax.net.ssl.TrustManager[] trustAllCerts = new
javax.net.ssl.TrustManager[]{
                new javax.net.ssl.X509TrustManager() {
                    public java.security.cert.X509Certificate[] getAcceptedIssuers() {
                        return null;
                    }
                    public void checkClientTrusted(java.security.cert.X509Certificate[]
certs, String authType) {
                    }
                    public void checkServerTrusted(java.security.cert.X509Certificate[]
certs, String authType) {
                    }
                }
            };
            javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("SSL");
            sc.init(null, trustAllCerts, new java.security.SecureRandom());

javax.net.ssl.HttpsURLConnection.setDefaultSSLSocketFactory(sc.getSocketFactory());
            javax.net.ssl.HttpsURLConnection.setDefaultHostnameVerifier((hostname,
session) -> true);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

}
```

Output: Appended in the final java class Main.

```
// ============================================================
// Step 4: Canonical SMILES
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.smiles.SmilesGenerator;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.exception.InvalidSmilesException;

public class Step4_Canonicalize {
    public static void main(String[] args) {

System.out.println("============================================================");
        System.out.println("  STEP 4 — Canonical SMILES Normalization");

System.out.println("============================================================\n");

        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);

        // CDK's canonical SMILES generator (unique() applies Morgan algorithm)
        SmilesGenerator smiGen = SmilesGenerator.unique();
```

```
        // ── Part A: Show that different SMILES give the same canonical form ──
        System.out.println("  PART A: Different inputs -> same canonical SMILES\n");
        System.out.println("  Compound: Acetic acid");
        System.out.println();

        String[] aceticAcidVariants = {
            "CC(=O)O", // standard textbook notation
            "OC(C)=O", // reverse traversal
            "C(C)(=O)O", // explicit branch
            "CC(O)=O", // O before =O
        };

        System.out.printf("  %-28s  %-30s  %s%n",
                "Input SMILES", "Canonical SMILES", "Valid?");
        System.out.println("  " + "-".repeat(70));

        for (String smi : aceticAcidVariants) {
            try {
                IAtomContainer mol = parser.parseSmiles(smi);
                AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                String canonical = smiGen.create(mol);
                System.out.printf("  %-28s  %-30s  YES%n", smi, canonical);
            } catch (Exception e) {
                System.out.printf("  %-28s  %-30s  NO  (%s)%n",
                        smi, "(parse failed)", e.getMessage());
            }
        }

        // ── Part B: Normalize SMILES for key energetic compounds ──────────────
        System.out.println("\n  PART B: Canonical SMILES for energetic compounds");
        System.out.println("  Source: Klapötke (2019), Table 1.1\n");


        String[][] hemCompounds = {
            {"Nitromethane", "C[N+](=O)[O-]"},
            {"TNT", "[N+](=O)([O-])c1cc([N+](=O)[O-])cc(c1C)[N+](=O)[O-]"},
            {"RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]"},
            {"HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C2[N+](=O)[O-
]1"},
            {"PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-
]"},
            {"Picric acid", "c1c(O)c([N+](=O)[O-])cc([N+](=O)[O-])c1[N+](=O)[O-]"},
            {"FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N"},
            {"NTO", "O=c1[nH]nno1"},
            {"TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]"},
            {"Ammonium nitrate", "[NH4+].[O-][N+]([O-])=O"},};

        System.out.printf("  %-22s  %-14s  %s%n",
                "Name", "Formula", "Canonical SMILES");
        System.out.println("  " + "-".repeat(80));

        for (String[] entry : hemCompounds) {
            String name = entry[0];
            String raw = entry[1];
            try {
                IAtomContainer mol = parser.parseSmiles(raw);
                AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);

                IMolecularFormula formula
                        = MolecularFormulaManipulator.getMolecularFormula(mol);
                String formulaStr
                        = MolecularFormulaManipulator.getString(formula);
                String canonical = smiGen.create(mol);
                // Truncate long SMILES for display
                String display = canonical.length() > 42
```

```
                        ? canonical.substring(0, 42) + "..."
                        : canonical;
                System.out.printf("  %-22s  %-14s  %s%n", name, formulaStr, display);
            } catch (Exception e) {
                System.out.printf("  %-22s  %-14s  ERROR: %s%n",
                        name, "?", e.getMessage());
            }
        }
        // ── Part C: Demonstrate stereo-specific SMILES ──────────────────────

        SmilesGenerator isoGen = SmilesGenerator.isomeric();
        String[] stereoSmiles = {
            "C[C@@H](O)N", // L-alaninol — stereo specified
            "C[C@H](O)N", // D-alaninol — opposite stereo
        };

        System.out.printf("  %-22s  %-22s  %s%n",
                "Input", "Canonical (no stereo)", "Isomeric (stereo)");
        System.out.println("  " + "-".repeat(70));

        for (String smi : stereoSmiles) {
            try {
                IAtomContainer mol = parser.parseSmiles(smi);
                AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
                String canonical = smiGen.create(mol);
                String isomeric = isoGen.create(mol);
                System.out.printf("  %-22s  %-22s  %s%n", smi, canonical, isomeric);
            } catch (Exception e) {
                System.out.printf("  %-22s  ERROR: %s%n", smi, e.getMessage());
            }
        }

    }
}


// ============================================================
//  Step 5: InChI and InChIKey
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================
package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.smiles.SmilesGenerator;
import org.openscience.cdk.inchi.InChIGeneratorFactory;
import org.openscience.cdk.inchi.InChIGenerator;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;
import org.openscience.cdk.interfaces.IMolecularFormula;

import java.util.LinkedHashMap;
import java.util.Map;

public class Step5_InChI {

    public static class MoleculeIdentifiers {

        public final String name;
        public final String smiles;
        public final String canonicalSmiles;
        public final String formula;
        public final double molecularWeight;
        public final String inchi;
```

```
    public final String inchiKey;
    public final String inchiStatus;   // "OK", "WARNING", "ERROR"

    public MoleculeIdentifiers(String name, String smiles, String canonical,
            String formula, double mw,
            String inchi, String inchiKey, String status) {
        this.name = name;
        this.smiles = smiles;
        this.canonicalSmiles = canonical;
        this.formula = formula;
        this.molecularWeight = mw;
        this.inchi = inchi;
        this.inchiKey = inchiKey;
        this.inchiStatus = status;
    }
}

// ── Core method: compute all identifiers from SMILES ──────────────────
public static MoleculeIdentifiers computeIdentifiers(
        String name, String smiles,
        SmilesParser parser,
        SmilesGenerator smiGen,
        InChIGeneratorFactory inchiFactory) {

    try {
        // 1. Parse SMILES
        IAtomContainer mol = parser.parseSmiles(smiles);
        AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
        AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);
        // 2. Canonical SMILES
        String canonical = smiGen.create(mol);
        // 3. Molecular formula and weight
        IMolecularFormula mfObj
                = MolecularFormulaManipulator.getMolecularFormula(mol);
        String formulaStr
                = MolecularFormulaManipulator.getString(mfObj);
        double mw
                = MolecularFormulaManipulator.getTotalExactMass(mfObj);
        // 4. InChI generation via CDK's JNA-InChI bridge
        InChIGenerator gen = inchiFactory.getInChIGenerator(mol);
        String inchiStr = gen.getInchi();
        String inchiKeyStr = gen.getInchiKey();
        String returnCode = gen.getReturnStatus().toString();
        return new MoleculeIdentifiers(name, smiles, canonical,
                formulaStr, mw,
                inchiStr, inchiKeyStr, returnCode);
    } catch (Exception e) {
        return new MoleculeIdentifiers(name, smiles, "ERROR", "?", 0.0,
                "ERROR: " + e.getMessage(), "?", "ERROR");
    }
}

public static void main(String[] args) {
    // ── Setup CDK objects ─────────────────────────────────────────────
    IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
    SmilesParser parser = new SmilesParser(builder);
    SmilesGenerator smiGen = SmilesGenerator.unique();
    InChIGeneratorFactory inchiFactory;
    try {
        inchiFactory = InChIGeneratorFactory.getInstance();
    } catch (Exception e) {
        System.err.println("ERROR: Could not initialize InChI factory.");
        System.err.println("  Make sure cdk-2.9.jar is on the classpath.");
        System.err.println("  Details: " + e.getMessage());
        return;
    }
    // ── Dataset: key HEMs from Klapötke (2019) ────────────────────────
```

```
        Map<String, String> compounds = new LinkedHashMap<>();
        compounds.put("Nitromethane", "C[N+](=O)[O-]");
        compounds.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        compounds.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
        compounds.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-
])C2[N+](=O)[O-]1");
        compounds.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-
])CO[N+](=O)[O-]");
        compounds.put("Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-
]");
        compounds.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
        compounds.put("NTO", "O=c1[nH]nno1");
        compounds.put("TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]");
        // ── Compute and display identifiers ──────────────────────────
        System.out.println("  Full identifier set for each compound:");
        System.out.println("  " + "=".repeat(60));

        for (Map.Entry<String, String> entry : compounds.entrySet()) {
            MoleculeIdentifiers ids
                    = computeIdentifiers(entry.getKey(), entry.getValue(),
                            parser, smiGen, inchiFactory);
            System.out.println();
            System.out.println("  " + ids.name);
            System.out.println("  " + "-".repeat(50));
            System.out.printf("    Formula         : %s%n", ids.formula);
            System.out.printf("    Exact MW        : %.5f g/mol%n",
ids.molecularWeight);
            System.out.printf("    Input SMILES    : %s%n", ids.smiles);
            System.out.printf("    Canonical SMILES: %s%n", ids.canonicalSmiles);

            // Display InChI with line break after the formula layer for readability
            if (ids.inchi != null && ids.inchi.startsWith("InChI=")) {
                System.out.printf("    InChI           : %s%n", ids.inchi);
            } else {
                System.out.printf("    InChI           : %s%n", ids.inchi);
            }
            System.out.printf("    InChIKey        : %s%n", ids.inchiKey);
            System.out.printf("    InChI status    : %s%n", ids.inchiStatus);
        }

    }
}
```

Output: Appended in the final java class Main.

```
// ============================================================
//  Step 6: Draw Molecules
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.layout.StructureDiagramGenerator;
import org.openscience.cdk.depict.DepictionGenerator;
import org.openscience.cdk.renderer.color.UniColor;

import java.awt.Color;
import java.io.File;
import java.util.ArrayList;
import java.util.LinkedHashMap;
```

```
import java.util.List;
import java.util.Map;

public class Step6_DrawMolecules {

    // ── Energetic compound dataset ──────────────────────────────────
    // Listed in order of increasing complexity (Klapötke 2019, Chapter 1)
    private static final Map<String, String> COMPOUNDS = new LinkedHashMap<>();

    static {
        COMPOUNDS.put("Nitromethane", "C[N+](=O)[O-]");
        COMPOUNDS.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        COMPOUNDS.put("Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-
]");
        COMPOUNDS.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
        COMPOUNDS.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-
])C2[N+](=O)[O-]1");
        COMPOUNDS.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-
])CO[N+](=O)[O-]");
        COMPOUNDS.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
        COMPOUNDS.put("TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]");
        COMPOUNDS.put("NTO", "O=c1[nH]nno1");
    }

    public static void main(String[] args) {


System.out.println("============================================================");
        System.out.println("  STEP 6 — Draw Molecules to PNG Images");
        System.out.println("  CDK DepictionGenerator + StructureDiagramGenerator");

System.out.println("============================================================\n");

        // ── Create output directories ──────────────────────────────
        File outDir = new File("molecules");
        File indivDir = new File("molecules/individual_structures");
        File gridDir = new File("molecules/grid");

        outDir.mkdirs();
        indivDir.mkdirs();
        gridDir.mkdirs();

        // ── Setup CDK parser and diagram generator ─────────────────
        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);
        StructureDiagramGenerator sdg = new StructureDiagramGenerator();

        DepictionGenerator dg = new DepictionGenerator()
                .withAtomColors() // CPK colors: C=black, N=blue, O=red, H=grey
                .withBackgroundColor(Color.WHITE)
                .withPadding(0.5) // padding around structure (bond-length units)
                .withTerminalCarbons() // show terminal CH3 carbon labels
                .withAromaticDisplay();     // draw aromatic rings with circle notation

        // ── Process each compound ──────────────────────────────────
        List<IAtomContainer> molList = new ArrayList<>();
        List<String> labelList = new ArrayList<>();
        int successCount = 0;

        System.out.printf("  %-22s  %-10s  %s%n", "Compound", "Status", "Output file");
        System.out.println("  " + "-".repeat(65));

        for (Map.Entry<String, String> entry : COMPOUNDS.entrySet()) {
            String name = entry.getKey();
            String smiles = entry.getValue();
            String safeN = name.replace(" ", "_").replace("-", "_");
```

```
            try {
                // 1. Parse SMILES -> molecule graph
                IAtomContainer mol = parser.parseSmiles(smiles);
                AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);

                // 2. Generate 2D coordinates
                //    SMILES has no geometry; SDG computes a layout using
                //    template-matching and ring-system rules
                sdg.setMolecule(mol);
                sdg.generateCoordinates();
                IAtomContainer mol2D = sdg.getMolecule();

                // 3. Draw individual PNG (400 x 300 pixels)
                File outFile = new File(indivDir, safeN + ".png");
                dg.withSize(400, 300)
                        .depict(mol2D)
                        .writeTo(outFile.getAbsolutePath());

                System.out.printf("  %-22s  %-10s  %s%n",
                        name, "OK", outFile.getPath());

                // Collect for grid image
                molList.add(mol2D);
                labelList.add(name);
                successCount++;

            } catch (Exception e) {
                System.out.printf("  %-22s  %-10s  ERROR: %s%n",
                        name, "FAILED", e.getMessage());
            }
        }

        // ── Draw grid image ──────────────────────────────────────────
        System.out.println();
        if (molList.size() >= 2) {
            try {
                File gridFile = new File(gridDir, "energetic_molecules_grid.png");

                // Convert list to array for CDK API
                IAtomContainer[] molArray = molList.toArray(new IAtomContainer[0]);
                String[] labelArray = labelList.toArray(new String[0]);

                // CDK grid depiction: 3 columns, auto rows
                dg.withSize(1200, 900) // total image size
                        .depict(molList, 3, -1) // 3 columns, auto rows
                        .writeTo(gridFile.getAbsolutePath());

                System.out.printf("  Grid image saved : %s%n", gridFile.getPath());

            } catch (Exception e) {
                System.out.println("  WARNING: Grid image generation failed: " +
e.getMessage());
                System.out.println("  Individual PNG files are still available.");
            }
        }
        // ── Display instructions for NetBeans ────────────────────────

        System.out.printf("  Processed: %d/%d compounds successfully%n",
                successCount, COMPOUNDS.size());

    }
}
```

Output: Appended in the final java class Main.

```
// ============================================================
//  Step 7: Molecular Formula and Descriptors
//  Chemoinformatics Tutorial — Chapter 1
//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;

import org.openscience.cdk.interfaces.IAtomContainer;
import org.openscience.cdk.interfaces.IAtom;
import org.openscience.cdk.interfaces.IChemObjectBuilder;
import org.openscience.cdk.interfaces.IMolecularFormula;
import org.openscience.cdk.interfaces.IIsotope;
import org.openscience.cdk.silent.SilentChemObjectBuilder;
import org.openscience.cdk.smiles.SmilesParser;
import org.openscience.cdk.tools.manipulator.AtomContainerManipulator;
import org.openscience.cdk.tools.manipulator.MolecularFormulaManipulator;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

public class Step7_MolecularFormula {

    // ── Data class for computed results ──────────────────────────────
    /**
     * Mirrors the dict produced by constitutional_descriptors() in Python.
     */
    public static class FormulaData {

        public final String name;
        public final String hillFormula;   // Hill notation (C then H then
alphabetical)
        public final double exactMW;       // g/mol (monoisotopic)
        public final double avgMW;         // g/mol (natural abundance average)
        public final int countC;
        public final int countH;
        public final int countN;
        public final int countO;
        public final double dbe;           // Double Bond Equivalents
        public final double nContentPct;   // Nitrogen content (wt%)
        public final double oContentPct;   // Oxygen content (wt%)
        public final double oxygenBalance; // OB% = (1600/MW)*(O - 2C - H/2)

        public FormulaData(String name, String hillFormula,
                double exactMW, double avgMW,
                int C, int H, int N, int O) {
            this.name = name;
            this.hillFormula = hillFormula;
            this.exactMW = exactMW;
            this.avgMW = avgMW;
            this.countC = C;
            this.countH = H;
            this.countN = N;
            this.countO = O;

            // DBE = (2C + 2 + N - H) / 2  for CxHyNzOw (O does not affect DBE)
            this.dbe = (2.0 * C + 2.0 + N - H) / 2.0;

            // Nitrogen content in weight percent
            this.nContentPct = (avgMW > 0) ? (14.007 * N / avgMW) * 100.0 : 0;

            // Oxygen content in weight percent
            this.oContentPct = (avgMW > 0) ? (15.999 * O / avgMW) * 100.0 : 0;

            // Oxygen balance for CaHbNcOd explosives (Klapötke 2019, Eq. 1.1)
```

```
            // OB% = (1600 / MW) * (d - 2a - b/2)
            this.oxygenBalance = (avgMW > 0)
                    ? (1600.0 / avgMW) * (O - 2.0 * C - H / 2.0)
                    : 0;
        }
    }

    // ── Core computation method ──────────────────────────────────────
    public static FormulaData computeFormulaData(
            String name, String smiles,
            SmilesParser parser) {

        try {
            IAtomContainer mol = parser.parseSmiles(smiles);
            AtomContainerManipulator.percieveAtomTypesAndConfigureAtoms(mol);
            // Convert implicit H to explicit so we can count them
            AtomContainerManipulator.convertImplicitToExplicitHydrogens(mol);

            // ── Molecular formula object ─────────────────────────────
            IMolecularFormula mf
                    = MolecularFormulaManipulator.getMolecularFormula(mol);

            String hillStr = MolecularFormulaManipulator.getString(mf);
            double exactMW = MolecularFormulaManipulator.getTotalExactMass(mf);
            double avgMW = MolecularFormulaManipulator.getTotalNaturalAbundance(mf);

            // ── Count atoms per element ──────────────────────────────
            // Iterate over explicit atoms; hydrogen is already explicit after
            // the convertImplicitToExplicitHydrogens call above
            Map<String, Integer> counts = new TreeMap<>();
            for (IAtom atom : mol.atoms()) {
                String sym = atom.getSymbol();
                counts.merge(sym, 1, Integer::sum);
            }

            int C = counts.getOrDefault("C", 0);
            int H = counts.getOrDefault("H", 0);
            int N = counts.getOrDefault("N", 0);
            int O = counts.getOrDefault("O", 0);

            return new FormulaData(name, hillStr, exactMW, avgMW, C, H, N, O);

        } catch (Exception e) {
            System.err.printf("  ERROR processing %s: %s%n", name, e.getMessage());
            return null;
        }
    }

    public static void main(String[] args) {

        // ── CDK setup ────────────────────────────────────────────────────
        IChemObjectBuilder builder = SilentChemObjectBuilder.getInstance();
        SmilesParser parser = new SmilesParser(builder);

        // ── Dataset with experimental detonation velocities ──────────────
        // Data source: Klapötke (2019), Table 1.1
        // Format: name -> {SMILES, experimental det. velocity m/s}
        Map<String, String> smiles = new LinkedHashMap<>();
        smiles.put("Nitromethane", "C[N+](=O)[O-]");
        smiles.put("TNT", "Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        smiles.put("Picric acid", "Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]");
        smiles.put("RDX", "C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]");
        smiles.put("HMX", "C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-
])C2[N+](=O)[O-]1");
        smiles.put("PETN", "C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-
])CO[N+](=O)[O-]");
        smiles.put("Nitroglycerin", "O([N+](=O)[O-])CC(O[N+](=O)[O-])CO[N+](=O)[O-]");
```

```
        smiles.put("FOX-7", "NC(=C([N+](=O)[O-])[N+](=O)[O-])N");
        smiles.put("TATB", "Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]");
        smiles.put("NTO", "O=c1[nH]nno1");
        smiles.put("Ammonium nitrate", "[NH4+].[O-][N+]([O-])=O");

        // ── Table 1: Basic formula data ──────────────────────────────────
        System.out.printf("  %-22s  %-14s  %8s  %4s  %4s  %4s  %4s%n",
                "Compound", "Formula", "Avg MW", "C", "H", "N", "O");
        System.out.println("  " + "-".repeat(75));

        java.util.List<FormulaData> allData = new java.util.ArrayList<>();

        for (Map.Entry<String, String> e : smiles.entrySet()) {
            FormulaData d = computeFormulaData(e.getKey(), e.getValue(), parser);
            if (d == null) {
                continue;
            }
            allData.add(d);
            System.out.printf("  %-22s  %-14s  %8.3f  %4d  %4d  %4d  %4d%n",
                    d.name, d.hillFormula, d.avgMW,
                    d.countC, d.countH, d.countN, d.countO);
        }

        // ── Table 2: Energetic descriptors ───────────────────────────────
        System.out.println();
        System.out.printf("  %-22s  %6s  %8s  %8s  %8s  Interpretation%n",
                "Compound", "DBE", "N wt%", "O wt%", "OB%");
        System.out.println("  " + "-".repeat(85));

        for (FormulaData d : allData) {
            String interp;
            if (d.oxygenBalance > 10) {
                interp = "over-oxidized";
            } else if (d.oxygenBalance > 0) {
                interp = "slight O excess";
            } else if (d.oxygenBalance > -10) {
                interp = "near-zero (high performance)";
            } else if (d.oxygenBalance > -40) {
                interp = "O-deficient";
            } else {
                interp = "severely O-deficient";
            }

            System.out.printf("  %-22s  %6.1f  %8.2f  %8.2f  %8.2f  %s%n",
                    d.name, d.dbe, d.nContentPct,
                    d.oContentPct, d.oxygenBalance, interp);
        }

    }
}
```

```java
// ============================================================
//  Main.java — Run All Chapter 1 Steps in Sequence
//  Chemoinformatics Tutorial — Chapter 1

//  Reference:
//  M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.
// ============================================================

package chemoinformatics;


public class Main {

    /** Width of the console separator line. */
    private static final int SEP_WIDTH = 62;

    public static void main(String[] args) {

        printBanner();

        // ── Step 1: Verify CDK Installation ──────────────────────
        printStepHeader(1, "Verify CDK Installation");
        try {
            Step1_VerifyInstall.main(args);
        } catch (Exception e) {
            printStepError(1, e);
        }

        // ── Step 2: SMILES Basics ────────────────────────────────
        printStepHeader(2, "SMILES Basics — Parse Energetic Molecules");
        try {
            Step2_SMILESBasics.main(args);
        } catch (Exception e) {
            printStepError(2, e);
        }

        // ── Step 3: Name to SMILES (PubChem) ─────────────────────
        printStepHeader(3, "Name to SMILES via PubChem REST API");
        System.out.println("  (This step requires an active internet connection.)");
        try {
            Step3_NameToSMILES.main(args);
        } catch (Exception e) {
            printStepError(3, e);
        }

        // ── Step 4: Canonical SMILES ─────────────────────────────
        printStepHeader(4, "Canonical SMILES Normalization");
        try {
            Step4_Canonicalize.main(args);
        } catch (Exception e) {
            printStepError(4, e);
        }

        // ── Step 5: InChI and InChIKey ───────────────────────────
        printStepHeader(5, "InChI and InChIKey Generation");
        try {
            Step5_InChI.main(args);
        } catch (Exception e) {
            printStepError(5, e);
        }

        // ── Step 6: Draw Molecules ───────────────────────────────
        printStepHeader(6, "Draw Molecules to PNG Images");
        try {
            Step6_DrawMolecules.main(args);
        } catch (Exception e) {
            printStepError(6, e);
```

```
        }

        // ── Step 7: Molecular Formula and Descriptors ──────────────────────
        printStepHeader(7, "Molecular Formula, MW, Atom Counts, Oxygen Balance");
        try {
            Step7_MolecularFormula.main(args);
        } catch (Exception e) {
            printStepError(7, e);
        }

        // ── Final summary ──────────────────────────────────────────────────
        printFinalSummary();
    }

    // ── Helper methods ─────────────────────────────────────────────────────

    private static void printBanner() {
        String line = "=".repeat(SEP_WIDTH);
        System.out.println(line);
        System.out.println("  CHEMOINFORMATICS TUTORIAL — CHAPTER 1");
        System.out.println("  Introduction and Environment Setup");
        System.out.println("  Java / NetBeans Edition");
        System.out.println();
        System.out.println("  Reference:");
        System.out.println("    Klapötke, T.M. (2019). Chemistry of High-Energy");
        System.out.println("    Materials, 4th ed. De Gruyter.");
        System.out.println();
        System.out.println("  Library: Chemistry Development Kit (CDK) 2.9");
        System.out.println("    Willighagen et al. (2017). J. Cheminform., 9, 33.");
        System.out.println(line);
        System.out.println();
    }

    private static void printStepHeader(int stepNum, String title) {
        System.out.println();
        System.out.println("╔" + "═".repeat(SEP_WIDTH - 2) + "╗");
        System.out.printf( "║  STEP %d: %-" + (SEP_WIDTH - 12) + "s║%n", stepNum,
title);
        System.out.println("╚" + "═".repeat(SEP_WIDTH - 2) + "╝");
        System.out.println();
    }

    private static void printStepError(int stepNum, Exception e) {
        System.out.println();
        System.out.printf("  [Step %d ERROR] %s: %s%n",
                          stepNum, e.getClass().getSimpleName(), e.getMessage());
        System.out.println("  Check that cdk-2.9.jar is in the lib/ folder and");
        System.out.println("  added to the NetBeans project Libraries.");
        System.out.println();
    }

    private static void printFinalSummary() {
        System.out.println();
        System.out.println("=".repeat(SEP_WIDTH));
        System.out.println("  ALL STEPS COMPLETE — CHAPTER 1 TUTORIAL");
        System.out.println("=".repeat(SEP_WIDTH));
        System.out.println();
        System.out.println("  What you have learned:");
        System.out.println("    Step 1: How to verify CDK is working in NetBeans");
        System.out.println("    Step 2: SMILES notation and molecule parsing");
        System.out.println("    Step 3: Look up compounds by name (PubChem API)");
        System.out.println("    Step 4: Canonical SMILES — one unique form per
molecule");
        System.out.println("    Step 5: InChI and InChIKey — international
identifiers");
        System.out.println("    Step 6: 2D structure depiction with CDK");
        System.out.println("    Step 7: Molecular formula, MW, atom counts, OB%");
```

```
        System.out.println();
        System.out.println("  Next: Chapter 2 — 3D Structures and Geometry");
        System.out.println("        (see Part2_Structures_and_3D_Geometry.md)");
        System.out.println();
        System.out.println("  Key references:");
        System.out.println("    1. Klapötke (2019). Chem. High-Energy Materials. De
Gruyter.");
        System.out.println("    2. Willighagen et al. (2017). CDK v2.0. J. Cheminform.
9,33.");
        System.out.println("    3. Weininger (1988). SMILES. J. Chem. Inf. Sci.
28,31.");
        System.out.println("    4. Heller et al. (2015). InChI. J. Cheminform. 7,23.");
        System.out.println("    5. Kim et al. (2023). PubChem. NAR 51, D1373.");
        System.out.println("=".repeat(SEP_WIDTH));
    }
}
```

Output:

```
run:
==
  CHEMOINFORMATICS TUTORIAL  CHAPTER 1
  Introduction and Environment Setup
  Java / NetBeans Edition

  Reference:
    M.Karthikeyan, Renu Vyas (2014). Practical Chemoinformatics, Springer.

  Library: Chemistry Development Kit (CDK) 2.9
    Willighagen et al. (2017). J. Cheminform., 9, 33.
==

  CDK version   : 2.12
  SMILES parse  : OK  RDX parsed successfully
  Heavy atoms   : 15 (expected: 15 heavy atoms)

  Checking required CDK classes:
    SmilesParser                    [OK]
    SmilesGenerator                 [OK]
    MolecularFormulaManipulator     [OK]
    CircularFingerprinter           [OK]
    WeightDescriptor                [OK]

  All CDK imports OK  environment is ready.
  Java version  : 24.0.1
  OS            : Windows 11

  Proceed to Step 2: SMILES Basics


  STEP 2  SMILES Basics
  What SMILES looks like for energetic molecules

  SMILES NOTATION RULES:
  ----------------------
  Atoms      : C  N  O  H  S  P  (element symbols)
  Double bond: =    e.g.  C=O  (carbonyl)
  Triple bond: #    e.g.  C#N  (nitrile)
  Branch     : ()   e.g.  CC(=O)O  (acetic acid)
  Ring       : num  e.g.  c1ccccc1  (benzene)
  Charge     : [NH4+]  [O-]  [N+]
  Aromatic   : lowercase  c  n  o

  Compound                Formula     Atoms     Parse Status
  ------------------------------------------------------------------------
  Methane                 CH4         5         OK
  Ethanol                 C2H6O       9         OK
  Acetic acid             C2H4O2      8         OK
  Benzene                 C6H6        12        OK
  Nitromethane            CH3NO2      7         OK
  TNT                     C7H5N3O6    21        OK
  RDX                     C3H6N6O6    21        OK
  HMX                     C5H9N8O8    30        OK
  PETN                    C5H8N4O12   29        OK
  Ammonium nitrate        H4N2O3      9         OK
  Picric acid             C6H3N3O7    19        OK
  FOX-7                   C2H4N4O4    14        OK
  ------------------------------------------------------------------------
  Parsed successfully: 12 / 12 molecules

  NITRO GROUP ENCODING NOTE:
  The -NO2 group is written as [N+](=O)[O-] in SMILES.
  This zwitterionic form is chemically correct and is the
  standard representation used by CDK, RDKit, and PubChem.
```

Klap□tke (2019), Section 1.1 □ structural conventions.

Proceed to Step 3: Name-to-SMILES lookup via PubChem.

(This step requires an active internet connection.)

Name to SMILES via PubChem REST API (Java 1.8)
Requires: active internet connection

```
Searching Pubchem: TNT
Searching Pubchem: RDX
Searching Pubchem: HMX
Searching Pubchem: PETN
Searching Pubchem: nitromethane
Searching Pubchem: ammonium nitrate
Searching Pubchem: picric acid
Searching Pubchem: tetryl
Searching Pubchem: TATB
Searching Pubchem: FOX-7
Compound Name     CID   Formula     Mol Wt      SMILES      InChI InChIKey    Atom
count Bond Count
TNT   8376  C7H5N3O6    227.13      CC1=C(C=C(C=C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-
]     InChI=1S/C7H5N3O6/c1-4-6(9(13)14)2-5(8(11)12)3-7(4)10(15)16/h2-3H,1H3
      SPSSULHKWOKEEL-UHFFFAOYSA-N   16    16
RDX   8490  C3H6N6O6    222.12      C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
      InChI=1S/C3H6N6O6/c10-7(11)4-1-5(8(12)13)3-6(2-4)9(14)15/h1-3H2   XTFIVUDBNACUBN-
UHFFFAOYSA-N      15    15
HMX   17596 C4H8N8O8    296.16      C1N(CN(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-
])[N+](=O)[O-]    InChI=1S/C4H8N8O8/c13-9(14)5-1-6(10(15)16)3-8(12(19)20)4-7(2-
5)11(17)18/h1-4H2 UZGLIIJVICEWHF-UHFFFAOYSA-N   20    20
PETN  6518  C5H8N4O12   316.14      C(C(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-
])O[N+](=O)[O-]   InChI=1S/C5H8N4O12/c10-6(11)18-1-5(2-19-7(12)13,3-20-8(14)15)4-21-
9(16)17/h1-4H2    TZRXHJWUDPFEEY-UHFFFAOYSA-N   21    20
nitromethane      6375  CH3NO2      61.040      C[N+](=O)[O-]     InChI=1S/CH3NO2/c1-
2(3)4/h1H3  LYGJENNIWJXYER-UHFFFAOYSA-N   4     3
tetryl      10178 C7H5N5O8    287.14      CN(C1=C(C=C(C=C1[N+](=O)[O-])[N+](=O)[O-
])[N+](=O)[O-])[N+](=O)[O-]   InChI=1S/C7H5N5O8/c1-8(12(19)20)7-5(10(15)16)2-
4(9(13)14)3-6(7)11(17)18/h2-3H,1H3  AGUIVNYEYSCPNI-UHFFFAOYSA-N   20    20
TATB  18286 C6H6N6O6    258.15      C1(=C(C(=C(C(=C1[N+](=O)[O-])N)[N+](=O)[O-
])N)[N+](=O)[O-])N      InChI=1S/C6H6N6O6/c7-1-
4(10(13)14)2(8)6(12(17)18)3(9)5(1)11(15)16/h7-9H2     JDFUJAMTCCQARF-UHFFFAOYSA-N   18
      18
FOX-7 536770      C2H4N4O4    148.08      C(=C([N+](=O)[O-])[N+](=O)[O-])(N)N
      InChI=1S/C2H4N4O4/c3-1(4)2(5(7)8)6(9)10/h3-4H2  FUHQFAMVYDIUKL-UHFFFAOYSA-N   10
      9
======Not found in PubChem:=====
ammonium+nitrate
picric+acid


Citation:
1) Practical Chemoinformatics,2014, M.Karthikeyan, Renu Vyas;
2) Kim et al. (2023). PubChem 2023 update.
  Nucleic Acids Res., 51, D1373-D1380.
=======
```

STEP 4 □ Canonical SMILES Normalization

PART A: Different inputs -> same canonical SMILES

Compound: Acetic acid

Input SMILES	Canonical SMILES	Valid?
CC(=O)O	O=C(O)C	YES

```
  OC(C)=O                       O=C(O)C                       YES
  C(C)(=O)O                     O=C(O)C                       YES
  CC(O)=O                       O=C(O)C                       YES

  PART B: Canonical SMILES for energetic compounds
  Source: Klap□tke (2019), Table 1.1

  Name                    Formula         Canonical SMILES
  ------------------------------------------------------------------------------
  Nitromethane            CH3NO2          [H]C([H])([H])[N+](=O)[O-]
  TNT                     C7H5N3O6        [H]C1=C(C(=C(C([H])=C1[N+](=O)[O-])[N+](=O...
  RDX                     C3H6N6O6        [H]C1([H])N([N+](=O)[O-])C([H])([H])N([N+]...
  HMX                     C5H9N8O8        [H]C12N([N+](=O)[O-])C([H])([H])N([N+](=O)...
  PETN                    C5H8N4O12       [H]C([H])(O[N+](=O)[O-])C(C([H])([H])O[N+]...
  Picric acid             C6H3N3O7        [H]OC=1C(=C([H])C(=C(C1[H])[N+](=O)[O-])[N...
  FOX-7                   C2H4N4O4        [H]N([H])C(=C([N+](=O)[O-])[N+](=O)[O-])N(...
  NTO                     CHN3O2          [H]N1N=NOC1=O
  TATB                    C4H2N6O6        [H]N([H])C=1C(=NC(=NC1[N+](=O)[O-])[N+](=O...
  Ammonium nitrate        H4N2O3          [H][N+]([H])([H])[H].O=[N+]([O-])[O-]

  PART C: Isomeric vs. Canonical SMILES
  Isomeric SMILES encodes stereocenters (@ symbols).
  Canonical SMILES in CDK is non-isomeric by default.
  Use SmilesGenerator.isomeric() to preserve stereochemistry.

  Input                   Canonical (no stereo)   Isomeric (stereo)
  ------------------------------------------------------------------
  C[C@@H](O)N             OC(N)C                  C[C@@H](O)N
  C[C@H](O)N              OC(N)C                  C[C@H](O)N

  KEY POINT: Two molecules are structurally identical if and
  only if their canonical SMILES strings are equal.
  This is how chemical databases detect duplicates.

  STEP 5 □ InChI and InChIKey Generation


  Full identifier set for each compound:

WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by com.sun.jna.Native in an unnamed
module (file:/D:/ChemoinformaticsTutorial_NetBeans/ChemoinformaticsTutorial/lib/cdk-
2.12.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this
module
WARNING: Restricted methods will be blocked in a future release unless native access is
enabled


  Nitromethane
  --------------------------------------------------
    Formula          : CH3NO2
    Exact MW         : 61.01638 g/mol
    Input SMILES     : C[N+](=O)[O-]
    Canonical SMILES: [H]C([H])([H])[N+](=O)[O-]
    InChI            : InChI=1S/CH3NO2/c1-2(3)4/h1H3
    InChIKey         : LYGJENNIWJXYER-UHFFFAOYSA-N
    InChI status     : WARNING

  TNT
  --------------------------------------------------
    Formula          : C7H5N3O6
    Exact MW         : 227.01783 g/mol
    Input SMILES     : Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
    Canonical SMILES: [H]C1=C(C(=C(C([H])=C1[N+](=O)[O-])[N+](=O)[O-
])C([H])([H])[H])[N+](=O)[O-]
```

```
    InChI           : InChI=1S/C7H5N3O6/c1-4-6(9(13)14)2-5(8(11)12)3-7(4)10(15)16/h2-
3H,1H3
    InChIKey        : SPSSULHKWOKEEL-UHFFFAOYSA-N
    InChI status    : WARNING

  RDX
  ------------------------------------------------
    Formula         : C3H6N6O6
    Exact MW        : 222.03488 g/mol
    Input SMILES    : C1N(CN(CN1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
    Canonical SMILES: [H]C1([H])N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-
])C([H])([H])N1[N+](=O)[O-]
    InChI           : InChI=1S/C3H6N6O6/c10-7(11)4-1-5(8(12)13)3-6(2-4)9(14)15/h1-3H2
    InChIKey        : XTFIVUDBNACUBN-UHFFFAOYSA-N
    InChI status    : WARNING

  HMX
  ------------------------------------------------
    Formula         : C5H9N8O8
    Exact MW        : 309.05433 g/mol
    Input SMILES    : C1N2CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C2[N+](=O)[O-
]1
    Canonical SMILES: [H]C12N([N+](=O)[O-])C([H])([H])N([N+](=O)[O-
])C([H])([H])N([N+](=O)[O-])C([H])([H])N1C([H])([H])[O-][N+]2=O
    InChI           : InChI=1S/C5H9N8O8/c14-10-5-6(4-21-10)1-7(11(15)16)2-8(12(17)18)3-
9(5)13(19)20/h5H,1-4H2
    InChIKey        : NNOKJPWJUQOHDP-UHFFFAOYSA-N
    InChI status    : WARNING

  PETN
  ------------------------------------------------
    Formula         : C5H8N4O12
    Exact MW        : 316.01387 g/mol
    Input SMILES    : C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]
    Canonical SMILES: [H]C([H])(O[N+](=O)[O-])C(C([H])([H])O[N+](=O)[O-
])(C([H])([H])O[N+](=O)[O-])C([H])([H])O[N+](=O)[O-]
    InChI           : InChI=1S/C5H8N4O12/c10-6(11)18-1-5(2-19-7(12)13,3-20-8(14)15)4-
21-9(16)17/h1-4H2
    InChIKey        : TZRXHJWUDPFEEY-UHFFFAOYSA-N
    InChI status    : WARNING

  Picric acid
  ------------------------------------------------
    Formula         : C6H3N3O7
    Exact MW        : 228.99710 g/mol
    Input SMILES    : Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]
    Canonical SMILES: [H]OC=1C(=C([H])C(=C([H])C1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
    InChI           : InChI=1S/C6H3N3O7/c10-6-4(8(13)14)1-3(7(11)12)2-5(6)9(15)16/h1-
2,10H
    InChIKey        : OXNIZHLAWKMVMX-UHFFFAOYSA-N
    InChI status    : WARNING

  FOX-7
  ------------------------------------------------
    Formula         : C2H4N4O4
    Exact MW        : 148.02325 g/mol
    Input SMILES    : NC(=C([N+](=O)[O-])[N+](=O)[O-])N
    Canonical SMILES: [H]N([H])C(=C([N+](=O)[O-])[N+](=O)[O-])N([H])[H]
    InChI           : InChI=1S/C2H4N4O4/c3-1(4)2(5(7)8)6(9)10/h3-4H2
    InChIKey        : FUHQFAMVYDIUKL-UHFFFAOYSA-N
    InChI status    : WARNING

  NTO
  ------------------------------------------------
    Formula         : CHN3O2
    Exact MW        : 87.00688 g/mol
    Input SMILES    : O=c1[nH]nno1
```

```
  Canonical SMILES: [H]N1N=NOC1=O
  InChI           : InChI=1S/CHN3O2/c5-1-2-3-4-6-1/h(H,2,4,5)
  InChIKey        : QPFVYTIMLAFERT-UHFFFAOYSA-N
  InChI status    : OKAY

TATB
--------------------------------------------------
  Formula         : C4H2N6O6
  Exact MW        : 230.00358 g/mol
  Input SMILES    : Nc1c([N+](=O)[O-])nc([N+](=O)[O-])nc1[N+](=O)[O-]
  Canonical SMILES: [H]N([H])C=1C(=NC(=NC1[N+](=O)[O-])[N+](=O)[O-])[N+](=O)[O-]
  InChI           : InChI=1S/C4H2N6O6/c5-1-2(8(11)12)6-4(10(15)16)7-3(1)9(13)14/h5H2
  InChIKey        : VSVBDRDETCTKPK-UHFFFAOYSA-N
  InChI status    : WARNING


INCHI LAYER EXPLANATION:
InChI=1S / <formula> / c<connectivity> / h<hydrogen>
/q<charge> /p<proton> /b<dbl bond stereo> /t<sp3 stereo>

INCHIKEY FORMAT: XXXXXXXXXXXXXX-YYYYYYYYYY-Z
X (14 chars): hashed connectivity layer
Y (10 chars): hashed remaining layers
Z  (1 char) : protonation flag

Use the InChIKey to search:
-> PubChem  : pubchem.ncbi.nlm.nih.gov
-> Google   : search the InChIKey as a string
-> ChemSpider: chemspider.com

Citation: Heller et al. (2015). J. Cheminform., 7, 23.
```

STEP 6 □ Draw Molecules to PNG Images

```
CDK DepictionGenerator + StructureDiagramGenerator


Compound                Status      Output file
------------------------------------------------------------------
Nitromethane            OK          molecules\individual_structures\Nitromethane.png
TNT                     OK          molecules\individual_structures\TNT.png
Picric acid             OK          molecules\individual_structures\Picric_acid.png
RDX                     OK          molecules\individual_structures\RDX.png
HMX                     OK          molecules\individual_structures\HMX.png
PETN                    OK          molecules\individual_structures\PETN.png
FOX-7                   OK          molecules\individual_structures\FOX_7.png
TATB                    OK          molecules\individual_structures\TATB.png
NTO                     OK          molecules\individual_structures\NTO.png

WARNING: Grid image generation failed: Index 1 out of bounds for length 0
Individual PNG files are still available.

HOW TO VIEW IN NETBEANS:
1. In the Projects panel, right-click the project root
2. Select 'Refresh' to see the new 'molecules' folder
3. Double-click any .png file to open it in the viewer
4. Or: File menu -> Open File -> navigate to molecules/

Processed: 9/9 compounds successfully

Color scheme (CPK convention):
  Carbon   (C) : black
  Nitrogen (N) : dark blue
  Oxygen   (O) : red
  Hydrogen (H) : light grey (shown only on heteroatoms)

Depiction reference:
  CDK DepictionGenerator □ https://cdk.github.io
```

Structures from Klap□tke (2019), Figs. 1.1-1.8

STEP 7 □ Molecular Formula, MW, and Atom Counting

Compound	Formula	Avg MW	C	H	N	O
Nitromethane	CH3NO2	0.000	1	3	1	2
TNT	C7H5N3O6	0.000	7	5	3	6
Picric acid	C6H3N3O7	0.000	6	3	3	7
RDX	C3H6N6O6	0.000	3	6	6	6
HMX	C5H9N8O8	0.000	5	9	8	8
PETN	C5H8N4O12	0.000	5	8	4	12
Nitroglycerin	C3H5N3O9	0.000	3	5	3	9
FOX-7	C2H4N4O4	0.000	2	4	4	4
TATB	C4H2N6O6	0.000	4	2	6	6
NTO	CHN3O2	0.000	1	1	3	2
Ammonium nitrate	H4N2O3	0.000	0	4	2	3

Compound	DBE	N wt%	O wt%	OB%	Interpretation
Nitromethane	1.0	0.00	0.00	0.00	near-zero (high performance)
TNT	7.0	0.00	0.00	0.00	near-zero (high performance)
Picric acid	7.0	0.00	0.00	0.00	near-zero (high performance)
RDX	4.0	0.00	0.00	0.00	near-zero (high performance)
HMX	5.5	0.00	0.00	0.00	near-zero (high performance)
PETN	4.0	0.00	0.00	0.00	near-zero (high performance)
Nitroglycerin	3.0	0.00	0.00	0.00	near-zero (high performance)
FOX-7	3.0	0.00	0.00	0.00	near-zero (high performance)
TATB	7.0	0.00	0.00	0.00	near-zero (high performance)
NTO	3.0	0.00	0.00	0.00	near-zero (high performance)
Ammonium nitrate	0.0	0.00	0.00	0.00	near-zero (high performance)

OXYGEN BALANCE FORMULA (Klap□tke 2019, Eq. 1.1):
For compound CaHbNcOd with molecular weight MW (g/mol):
OB% = (1600 / MW) □ (d - 2a - b/2)

DOUBLE BOND EQUIVALENTS (DBE):
DBE = (2C + 2 + N - H) / 2
DBE = 0 -> fully saturated (alkane-like)
DBE = 1 -> one ring OR one double bond
DBE = 4 -> benzene ring
High DBE + N-rich + positive OB% -> energetic candidate

WORKED EXAMPLE □ RDX (C3H6N6O6, MW = 222.12 g/mol):
OB% = (1600/222.12) □ (6 - 2□3 - 6/2)
= (7.203) □ (6 - 6 - 3)
= (7.203) □ (-3)
= -21.6% [oxygen-deficient; produces some CO]
DBE = (2□3 + 2 + 6 - 6) / 2 = 4 / 2 = 2
-> 2 degrees of unsaturation (the 6-membered ring
counts as 1 ring + 3 C-N bonds, no pi bonds)

Experimental value for RDX OB%: -21.6% [matches calculation]
Source: Klap□tke (2019), Table 1.2

==
  ALL STEPS COMPLETE □ CHAPTER 1 TUTORIAL
==

  What you have learned:
    Step 1: How to verify CDK is working in NetBeans
    Step 2: SMILES notation and molecule parsing
    Step 3: Look up compounds by name (PubChem API)
    Step 4: Canonical SMILES □ one unique form per molecule
    Step 5: InChI and InChIKey □ international identifiers
    Step 6: 2D structure depiction with CDK
    Step 7: Molecular formula, MW, atom counts, OB%

  Next: Chapter 2 □ 3D Structures and Geometry
        (see Part2_Structures_and_3D_Geometry.md)

  Key references:
    1. Klap□tke (2019). Chem. High-Energy Materials. De Gruyter.
    2. Willighagen et al. (2017). CDK v2.0. J. Cheminform. 9,33.
    3. Weininger (1988). SMILES. J. Chem. Inf. Sci. 28,31.
    4. Heller et al. (2015). InChI. J. Cheminform. 7,23.
    5. Kim et al. (2023). PubChem. NAR 51, D1373.
==
BUILD SUCCESSFUL (total time: 11 seconds)

https://github.com/karthincl/hemscreener/tree/master/java